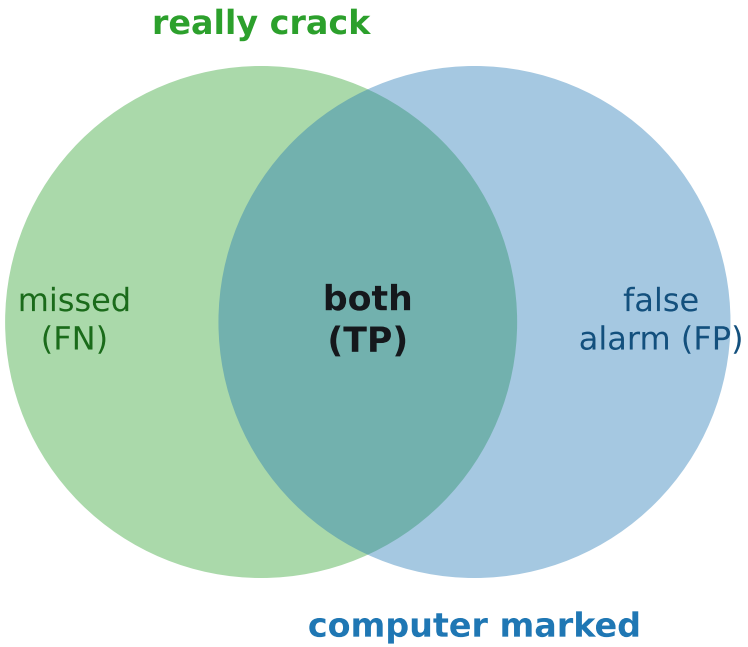
Crack Fractography — where the measurements stand

v1.49.2. Every figure here is generated from a committed record by a script in this repository; none is drawn by hand and none carries a number that was not measured.

Caveat. Eight slides. Read each one with its caveat: most of these results were over-read at some point for exactly the reason stated underneath.

All four scores are fractions of the same three areas

Green is the crack you painted. Blue is what the computer marked. Everything else is just which part of this picture you divide by.



Precision

both / everything BLUE

did it cry wolf?

Recall

both / everything GREEN

did it miss things?

F1

balances the two above

one number, not cheatable

IoU

both / the WHOLE picture above

strictest; rewards thickness

The words, in plain English (1 of 3)

Precision

Of everything the computer marked as crack, how much really was crack.

Watch out: High precision with low recall just means it only marked the easy bits.

Here: Yours on your tiles: 0.90 at a 2 px tolerance. Of what it marks, 9 in 10 is crack.

Recall

Of all the crack that is really there, how much the computer found.

Watch out: MARKING THE WHOLE IMAGE SCORES A PERFECT 1.0. Recall alone is never a score.

Here: Yours: 0.80. It finds four fifths of the crack you painted.

F1

The two above combined into one number, in a way that punishes being bad at either.

Watch out: You cannot cheat it by marking everything -- that destroys precision, so F1 collapses.

Here: Yours: 0.66. Marking everything would score 0.06.

The words, in plain English (2 of 3)

IoU (intersection over union)

The overlap divided by everything that either of you marked. The strictest of the four.

Watch out: It rewards drawing THICK. A perfectly placed thin line scores badly against a thick label, so a fatter, less accurate answer can beat a correct one.

Here: Yours: 0.50 at 2 px. A PERFECT answer caps at 0.50 here -- so you are near the limit, not at half marks.

clDice

Did you find the crack along its LENGTH -- following the path, ignoring how thick you drew it.

Watch out: On these labels it is still partly cheatable by thickening: a no-information dilation moved it +0.048 on 13 of 13 specimens.

Here: Yours: 0.64.

Skeleton-F1 at 2 px

The same idea as clDice but done properly: compare the two centre-lines, allowing a 2 pixel wobble.

Watch out: The one metric here that a thicker mask does NOT improve (+0.0015, p=0.68).

Here: Yours: 0.55. This is the honest headline number.

The words, in plain English (3 of 3)

Tolerance

How many pixels of slack before a pixel counts as wrong. '2 px' forgives being two pixels off.

Watch out: It changes the answer enormously, so a score without a stated tolerance is unreadable.

Here: On CrackForest the published F1 is 0.75 at 0 px and 0.97 at 2 px -- same model.

The floor (no-information level)

What a method that knows nothing would score, by marking at random.

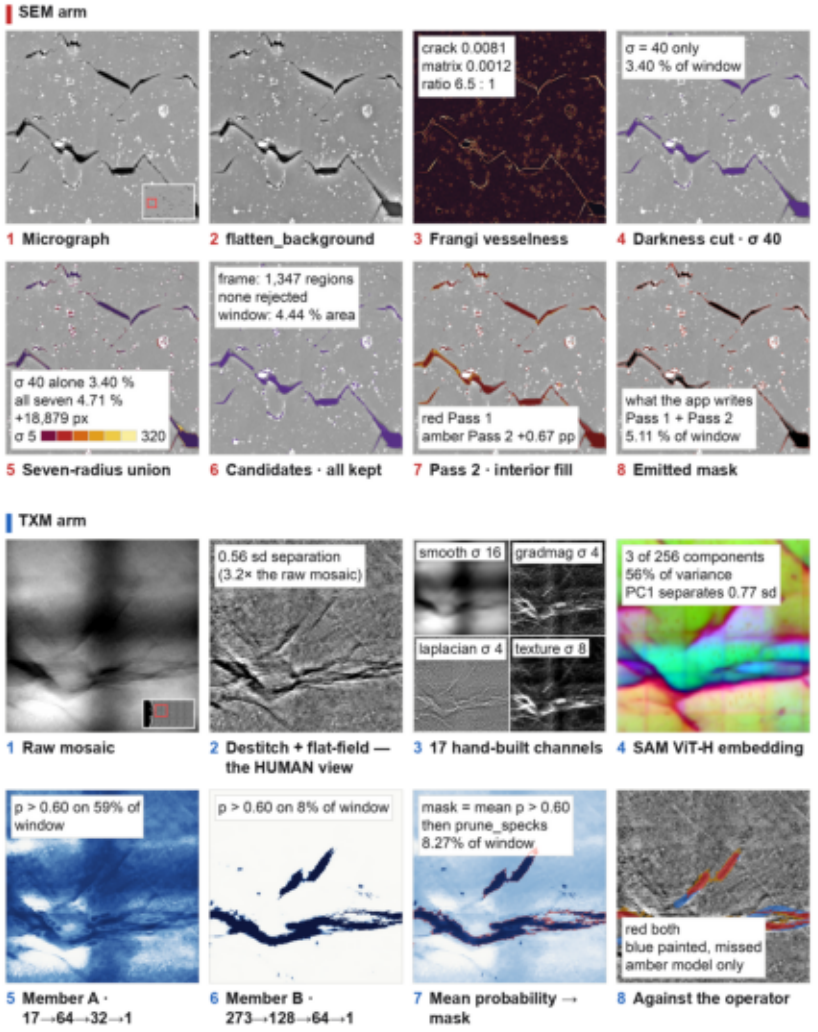
Watch out: A precision of 0.26 sounds poor; if the floor is 0.08 it is 3x better than guessing.

Here: Crack covers ~3% of your tiles, so random marking scores about 0.03.

What each stage of the detector does

The SEM row is v1.49.2, the pipeline the app runs: seven flattening radii unioned, every candidate region kept.

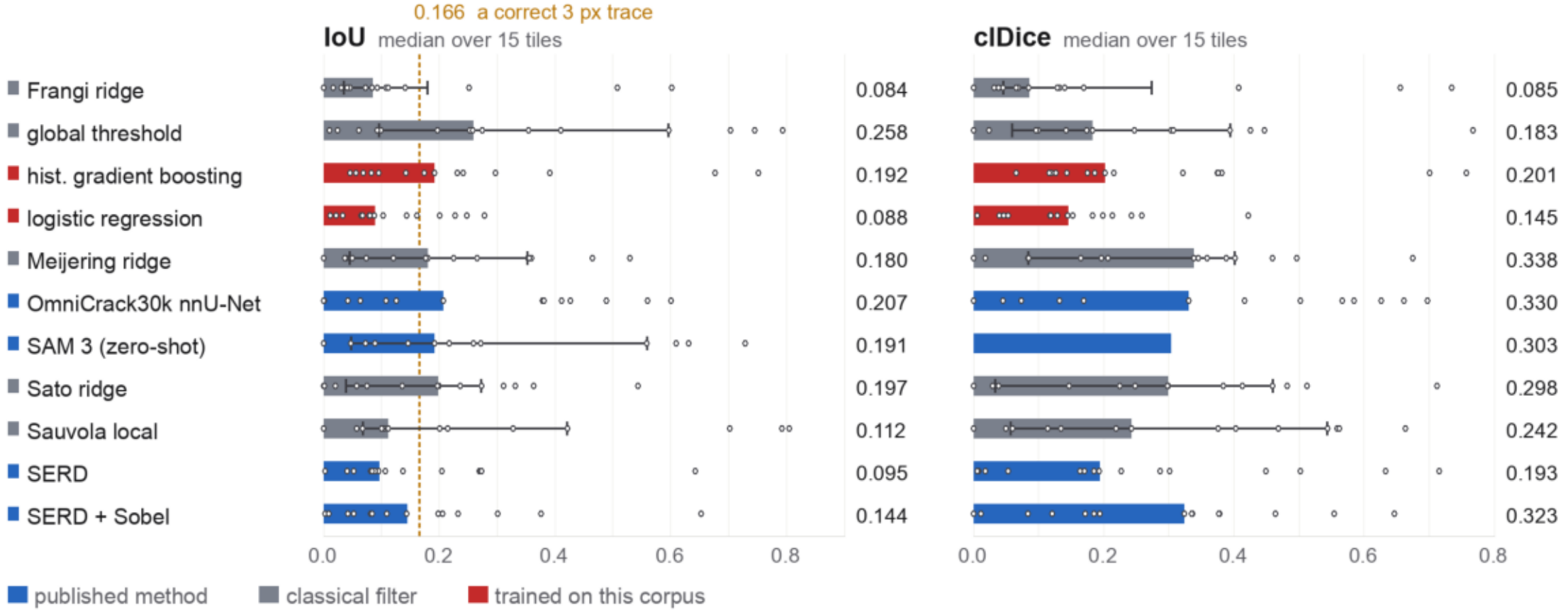
Caveat. Panel 6 is the candidate stage; panel 8 is the full pipeline including Pass 2. The retired region classifier would have removed 21.6% (207,448 px) of what now ships.



Every method on identical data

Eleven arms, the same 15 tiles from the same 9 frames, the same leave-one-frame-out budget. The ranking REVERSES between the two metrics, and that is the result.

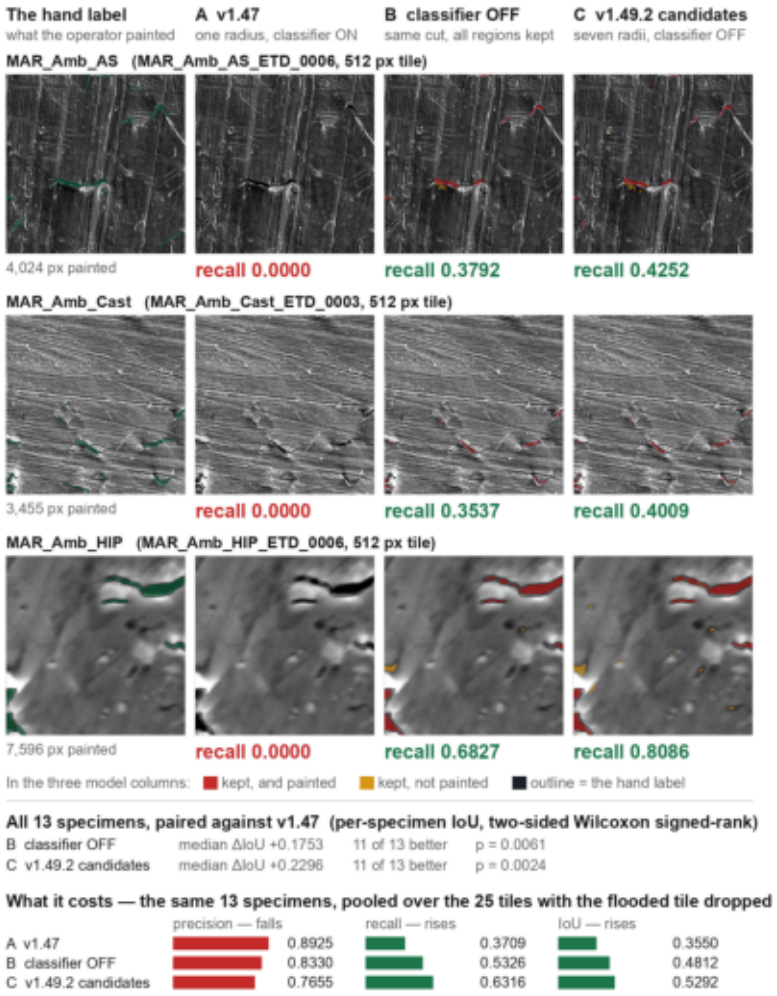
Caveat. The dashed line at IoU 0.166 is what a physically correct 3 px trace scores. Arms above it are not more accurate; they are fatter.



What deleting the region classifier recovers

Three ETD specimens scored recall 0.0000 at every candidate setting ever tried. The candidates covered those cracks all along; the classifier was rejecting them.

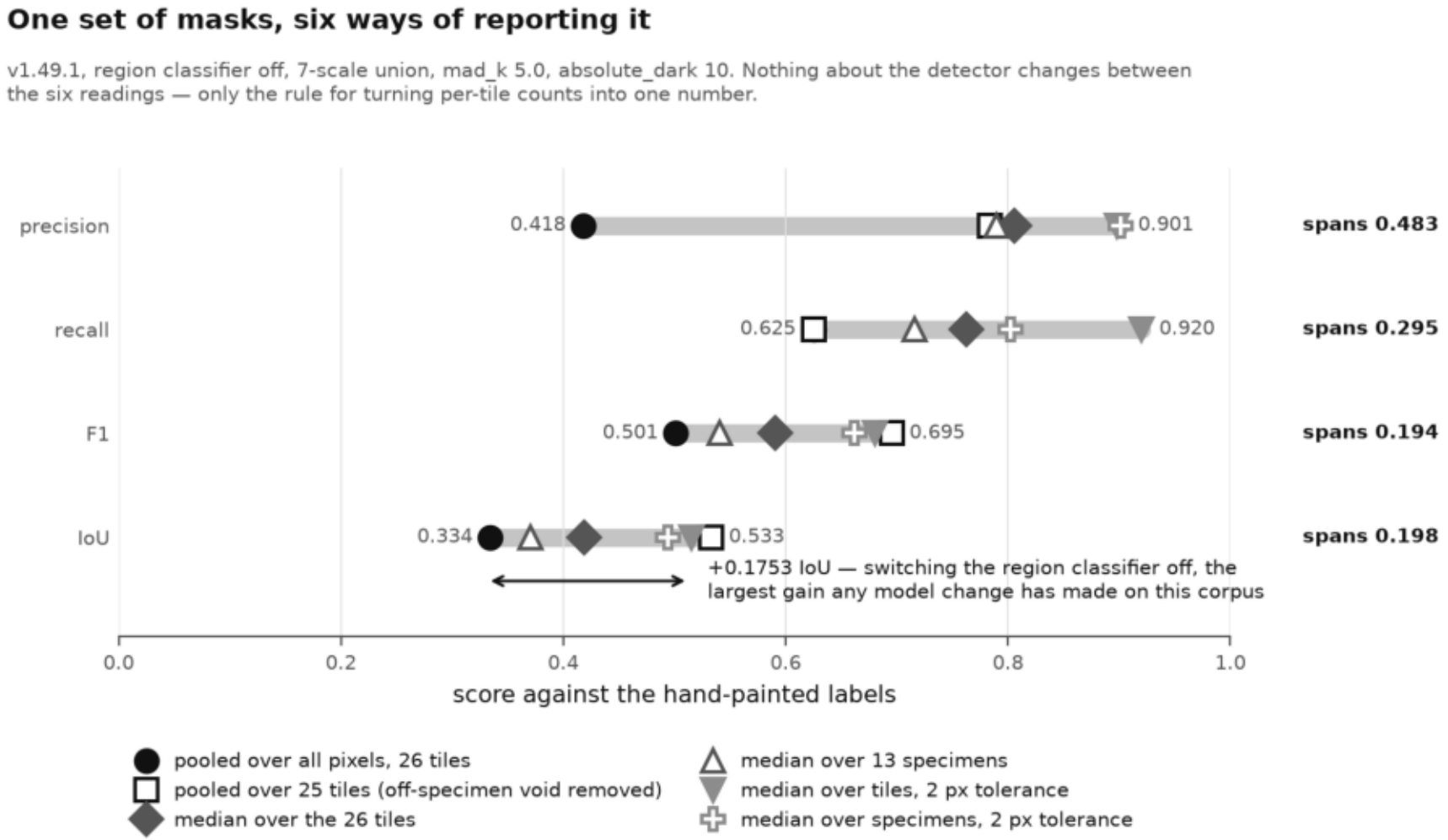
Caveat. It also COSTS precision on ordinary fields, 0.8925 to 0.7840 pooled off the one off-specimen void tile. Shown on the same plate.



The reporting choice moves the number more than any method does

One set of masks, six legitimate ways of reporting it. The IoU span across readings is 0.1985 -- larger than the +0.1753 that switching the classifier off actually bought.

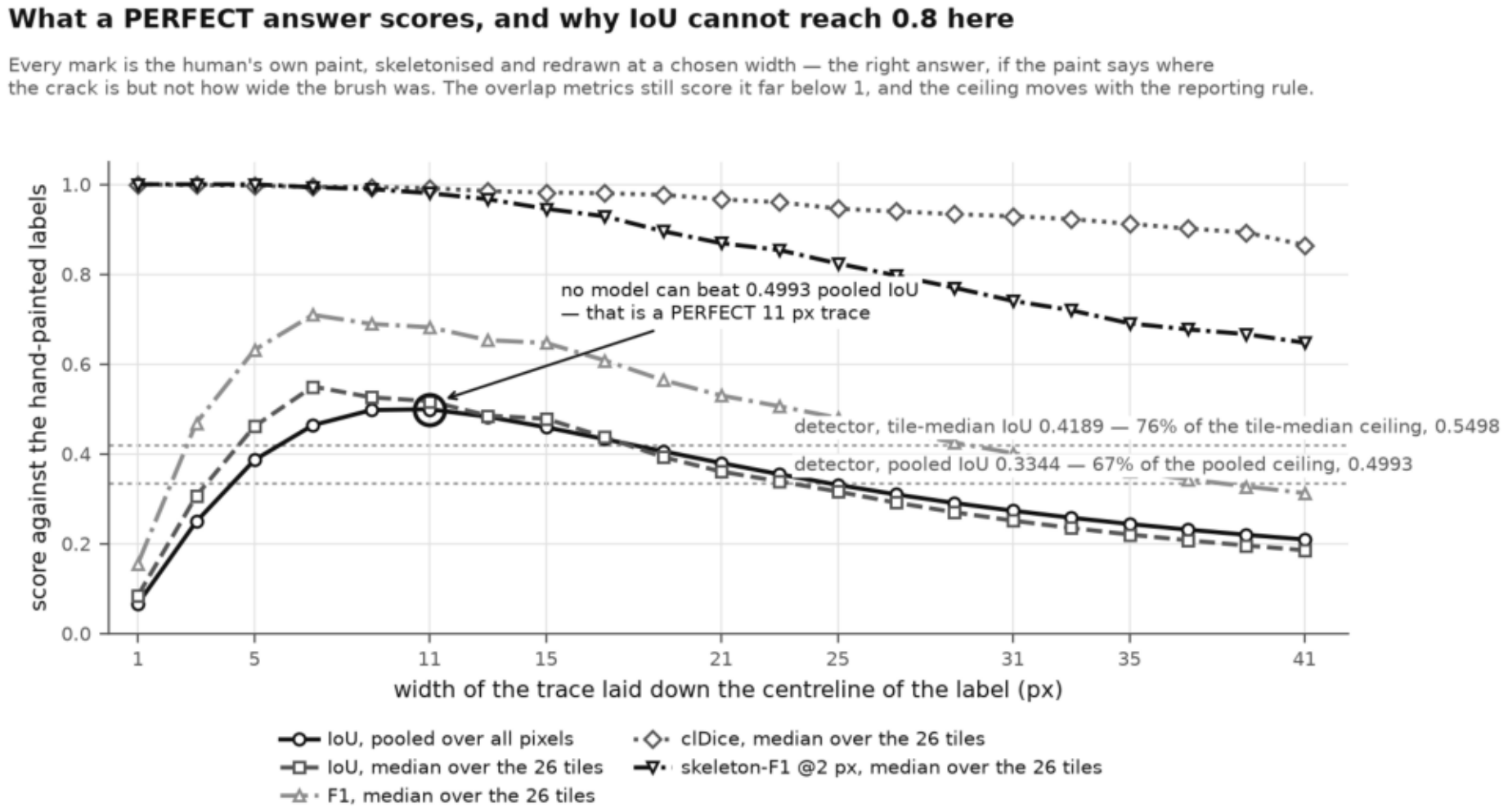
Caveat. Always state the aggregation AND the tolerance. The pooled row and the per-specimen-at-2px row describe the same pixels.



A correct answer caps at IoU 0.50 against these labels

A perfectly placed trace -- the label's own centreline, zero localisation error -- peaks at IoU 0.4993 pooled at an 11 px width, and 0.5498 as a median over tiles at 7 px.

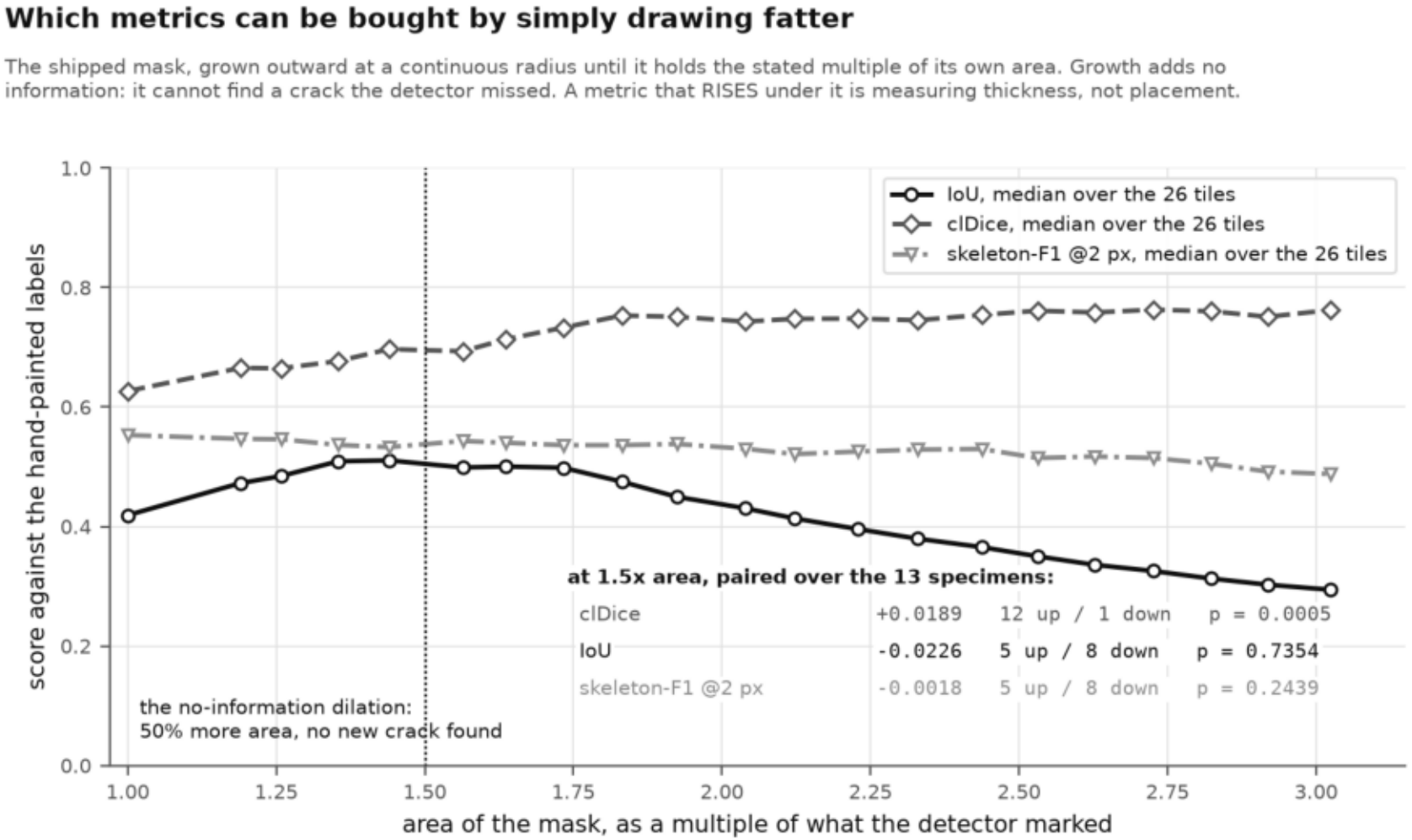
Caveat. So IoU 0.42 is 67-77% of what is achievable, not 42% of perfect. The ceiling is aggregation-dependent too and must be read in the same units as the score.



Both headline metrics reward being fatter than a crack

Dilating the mask to 1.5x its area adds no information and moves clDice +0.0478 on 13 of 13 specimens (p=0.0002). Skeleton-F1 at 2 px moves +0.0015 (p=0.68).

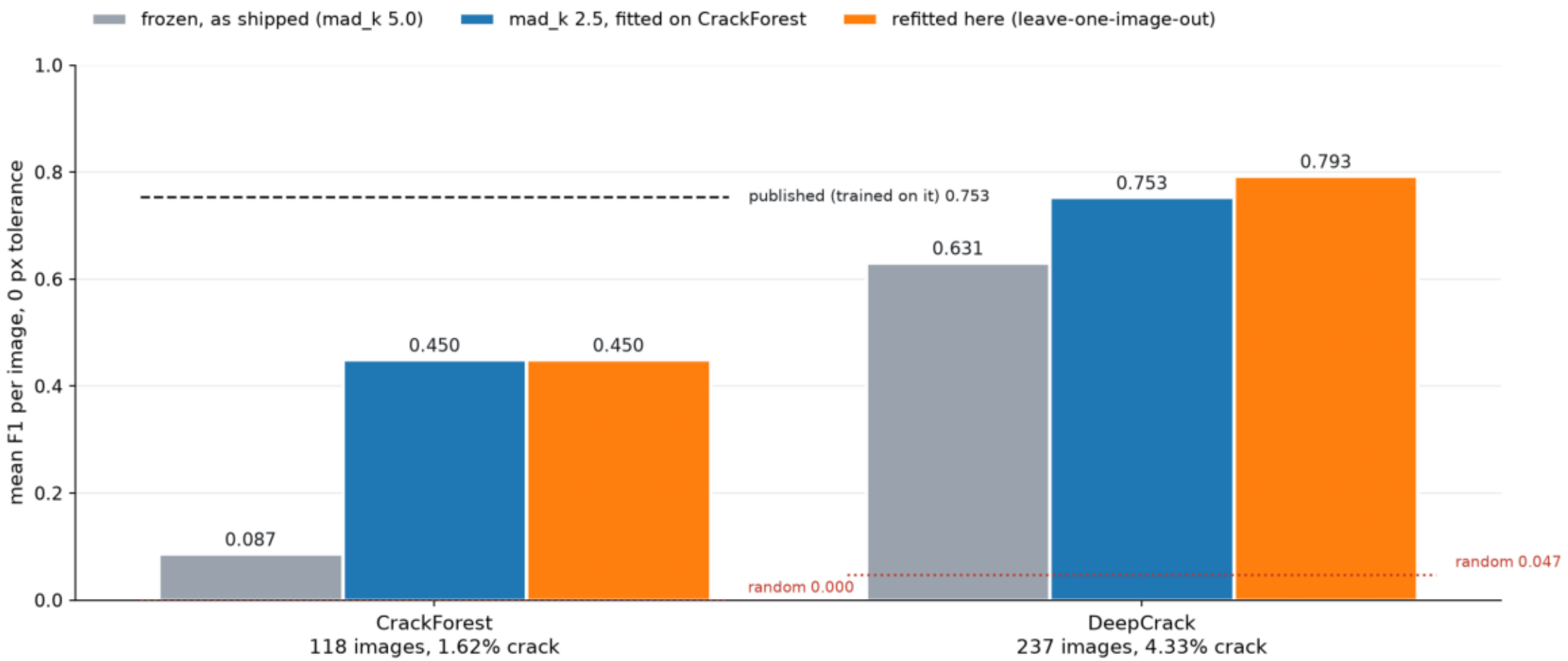
Caveat. The full-frame paint gate has the same disease: a correct 3 px trace of the human's own paint deletes 94.63% of it, because the paint is 154 px wide. Three instruments, one failure mode.



The same detector on public crack datasets

Frozen -- every constant as the app ships, nothing fitted -- it scores 0.631 on DeepCrack (237 images) and 0.087 on CrackForest (118). Refitting ONE constant, mad_k, takes those to 0.793 and 0.450.

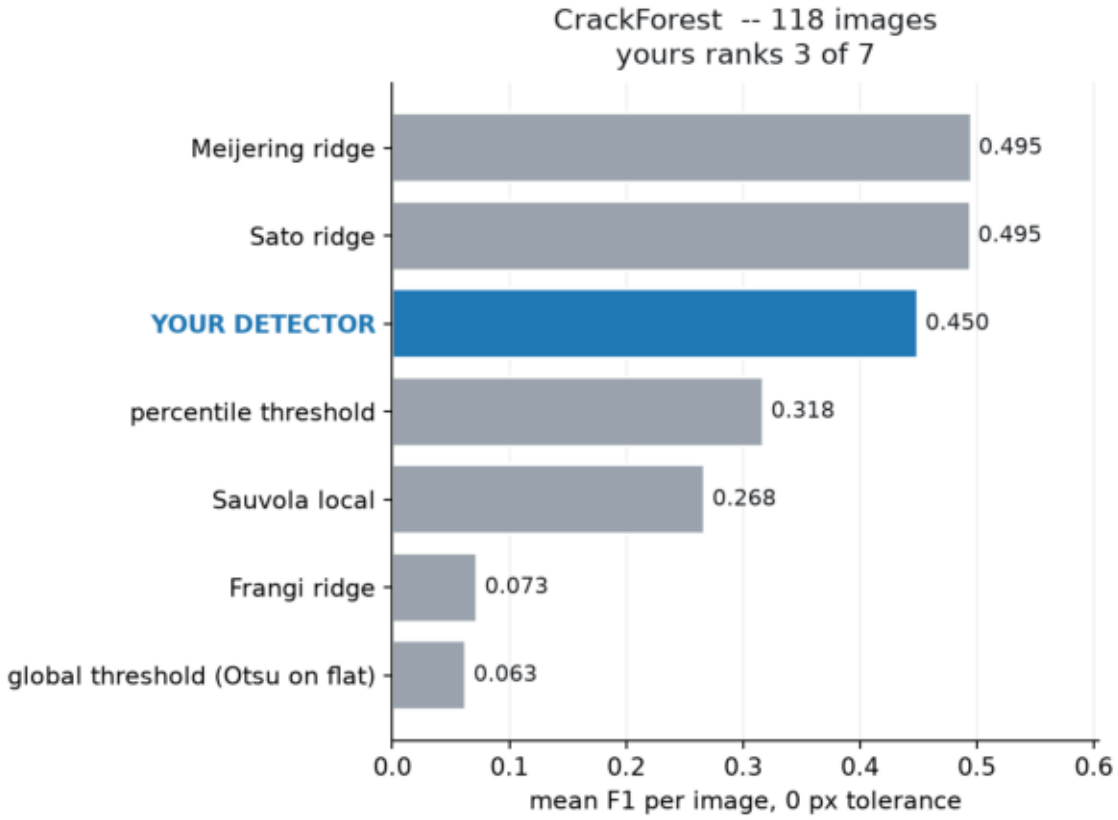
Caveat. CrackForest's fitted value carried BLIND to DeepCrack reaches 0.753, 95% of refitting there -- so one road-calibrated constant serves road imagery and the SEM-calibrated 5.0 is the outlier. Still below the 0.753 a model trained on CrackForest reports.

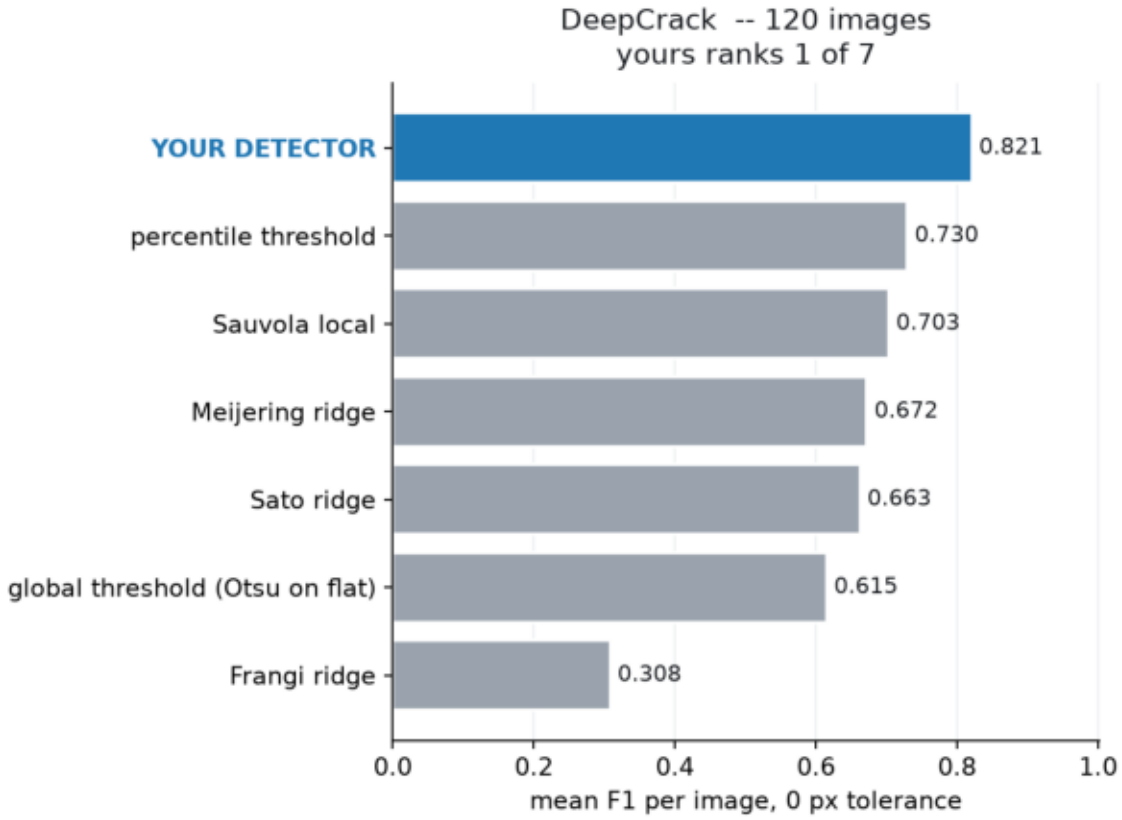


Against six classical baselines, on the same images

Every arm run here, same images, same scorer, and EVERY arm's parameter chosen leave-one-image-out so none is handicapped. It ranks 1 of 7 on DeepCrack (0.821 against 0.730) and 3 of 7 on CrackForest, behind the Meijering and Sato ridge filters at 0.495.

Caveat. So the claim is not 'better than classical methods' but 'better on wide high-contrast cracks, behind a ridge filter on thin faint ones' -- which is what a darkness detector should do. The two panels are 118 and 120 images; do not read across them.

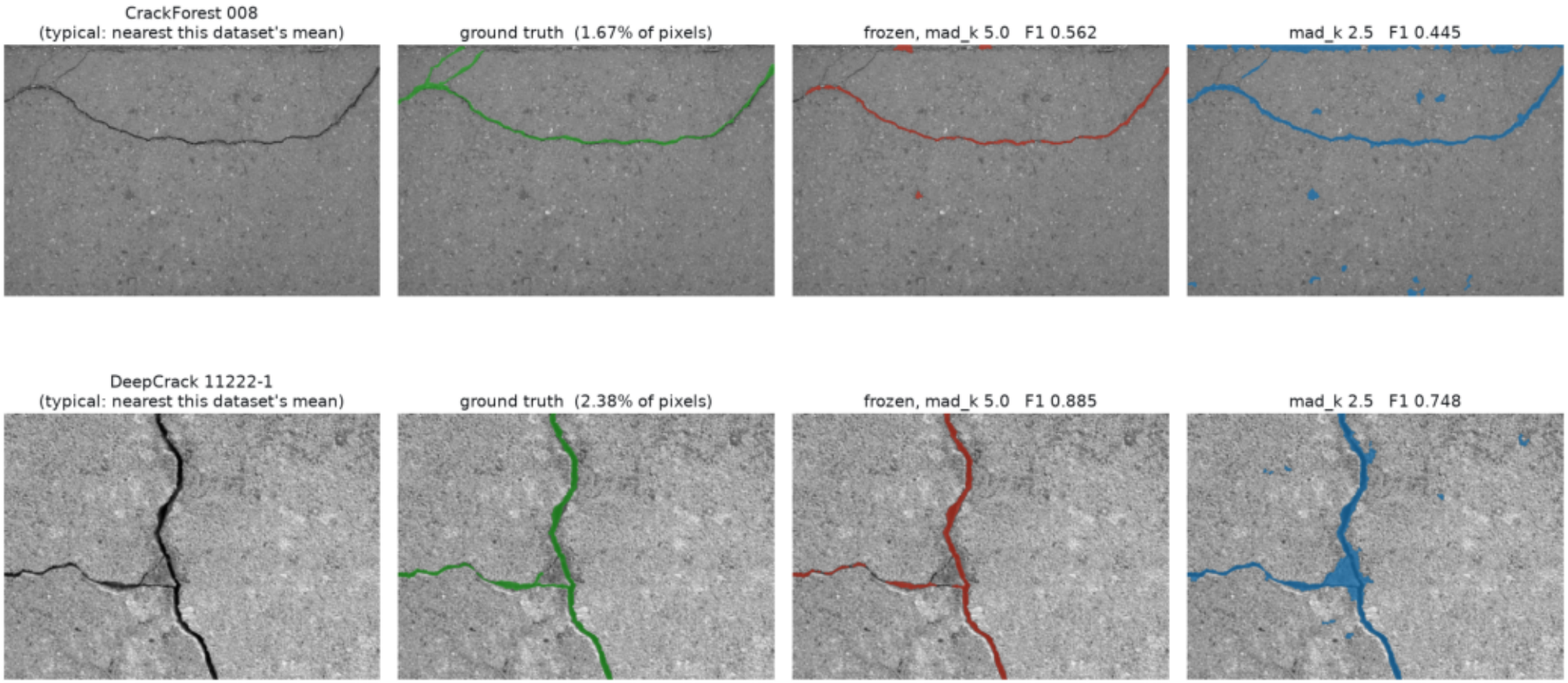




What that looks like on the public images

Ground truth against the frozen and refitted masks, on one typical image from each dataset.

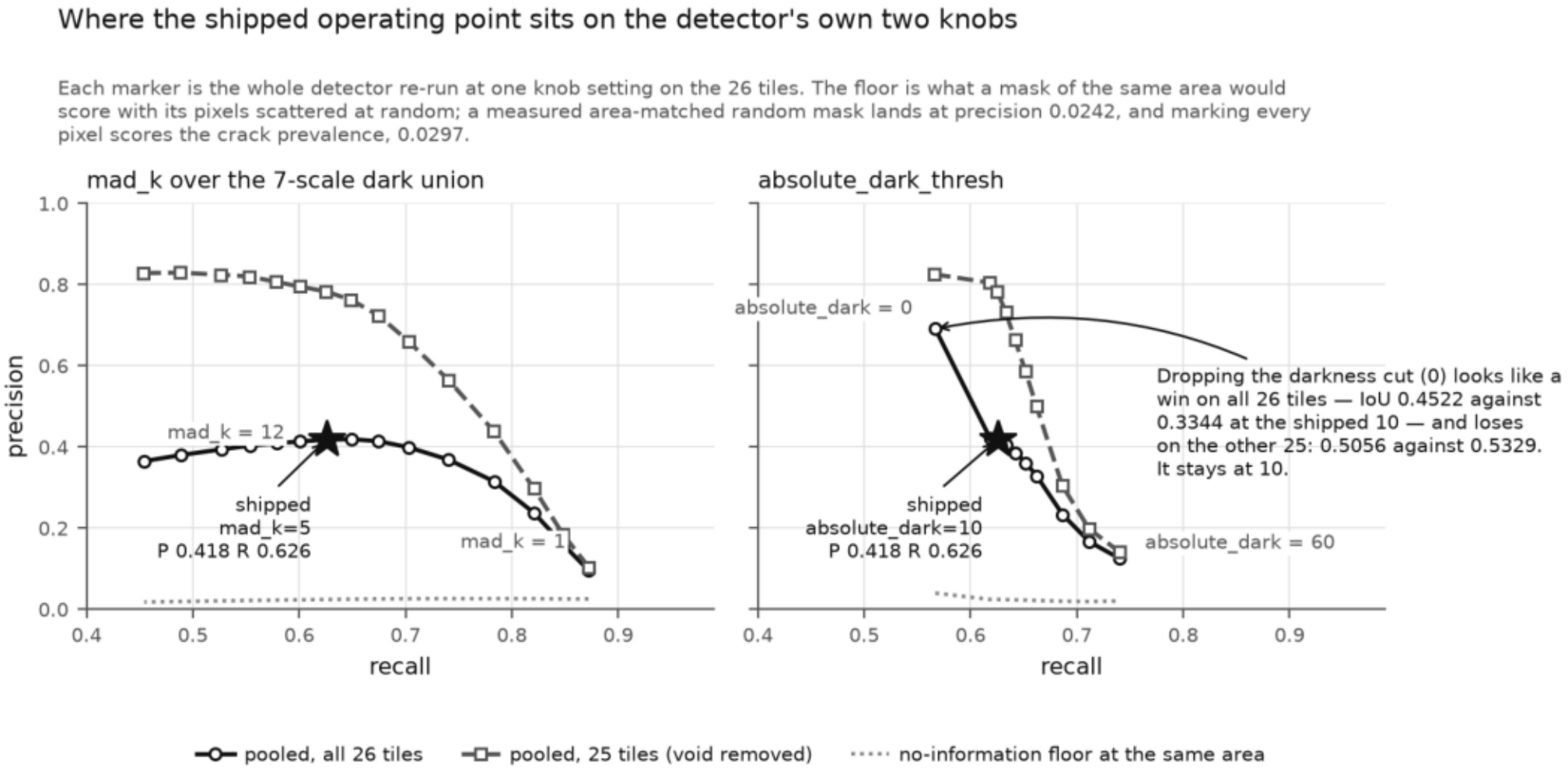
Caveat. Each image is chosen by RULE -- the one whose mad_k 2.5 score is nearest that dataset's mean -- not by eye.
The first draft hand-picked a CrackForest image where frozen beat the refit, the reverse of the aggregate.



Recall alone is not a score

The loosest setting reaches recall 0.98 and scores F1 0.162. Its precision, 0.0883, is identical to a plain Otsu threshold and is 1.09x a coin flip.

Caveat. Read precision against the no-information floor -- the prevalence of crack among refereed pixels, 0.0810 on TXM and 0.0297 on the SEM tiles.



What each measurement means (1 of 3)

Read straight out of app/groupstats.py, so this slide cannot drift from what the application shows.

Crack count

cracks

How many separate crack regions were found. A COUNT, NOT AN AMOUNT: one long crack and a hundred specks both add to it. Read it beside 'Largest crack's share of area'.

Largest crack's share of area

fraction of crack area

Of all the crack area in a frame, the fraction sitting in the single biggest region. Near 1.0 means one dominant crack; near 0 means a genuinely distributed network. This is what tells you whether the crack count means what you think.

Number density (per megapixel)

cracks / Mpx

Cracks per million pixels of image. Comparable only between frames at the SAME magnification, because a pixel is a different physical size at each.

Number density (per mm2, scale needed)

cracks / mm2

Cracks per square millimetre of specimen surface. The physical version of the row above, so it IS comparable across magnifications -- but only frames with a recovered pixel size contribute.

What each measurement means (2 of 3)

Read straight out of app/groupstats.py, so this slide cannot drift from what the application shows.

Total crack area (pixels)

px2

Every crack pixel in the group added up. Summing this across frames of different magnification adds areas that are not the same physical size.

Total crack area (um2, scale needed)

um2

The same total converted to physical area. Only frames with a known pixel size contribute, so this can cover fewer frames than the pixel row above it.

Crack area fraction

% of frame

What percentage of the imaged area is crack. Dimensionless, so it is the one size measure that compares fairly across magnifications and modalities.

Average crack area

px2

Mean area of one crack region. Pulled upward by a single large crack; compare it with the median below.

What each measurement means (3 of 3)

Read straight out of app/groupstats.py, so this slide cannot drift from what the application shows.

Median crack area

px2

The typical crack's area -- half are smaller, half larger. Unlike the mean, one huge crack cannot move it.

Average length (all)

px

Mean crack length along its centreline, INCLUDING cracks that run out of the frame. Those are measured only as far as the frame edge, so this is a lower bound.

Average length (uncensored only)

px

The same average over only those cracks fully inside the frame, so no length is cut short. If this is much larger than the row above, the frame is clipping your cracks.

Average aspect ratio

length / width

How elongated a region is: length divided by width. Around 1 is round -- porosity rather than a crack. Painted crack in this corpus runs about 3.3; painted porosity about 1.5.

How good is this model, against published ones?

On the 15-tile benchmark the published models were scored on, the shipped detector leads both metrics. The lead survives dropping the frames whose tiles informed its configuration, and does not survive dropping the specimens - n=2 there, which settles nothing either way.

Caveat. Every competing arm is scored leave-one-FRAME-out, which penalises it for having a tunable parameter. v1.49.2 has no parameter fitted on these tiles, but 3 of the 9 frames and 5 of 7 specimens also appear in the 26-tile set used to choose its configuration, so the all-15 row is the favourable reading. And at n=15 nothing on this benchmark separated statistically in the first place.

arm	IoU	clDice	basis
>>> shipped v1.49.2, all 15 ti	**0.3628**	**0.4246**	**no parameter fitted here; config infor**
>>> shipped v1.49.2, frame-o	**0.3530**	**0.3658**	**n=9; frames never used to choose the**
>>> shipped v1.49.2, specim	**0.1596**	**0.2883**	**n=2; too small to conclude, trends ag**
global threshold	0.2579	0.1827	one line of code, LOFO
NESTED LOFO ensemble	0.2526	0.1739	honest nested selection
Sato ridge	0.1972	0.2981	classical filter, LOFO
SAM 3 (zero-shot union)	0.1914	0.3030	foundation model, no fitting
OmniCrack30k nnU-Net	0.2069	0.3304	trained on 30,017 images incl. steel
Meijering ridge	0.1802	0.3382	classical filter, LOFO; best clDice of the cor
Sauvola local	0.1117	0.2421	classical filter, LOFO
Frangi ridge	0.0843	0.0849	classical filter, LOFO
a correct 3 px trace	0.1662	0.9997	the physically correct answer; IoU ceiling r

The pipeline, one real image per stage

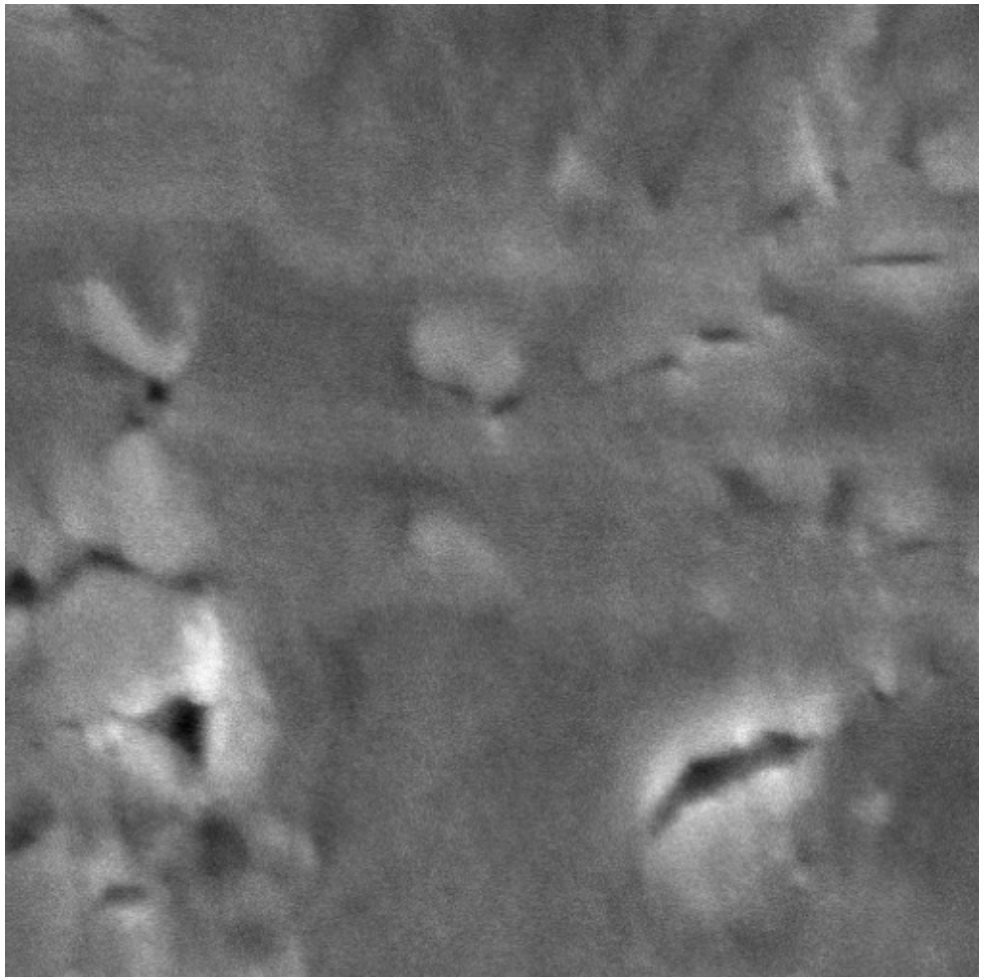
Frame HIP_24hr_SE_Side_006, window 262,144 px. Every panel is a real array from the real pipeline -- no mock-ups, no schematic boxes.

Caveat. The frame is chosen by a rule that works against the detector: the lower of the two central tiles by IoU, tie broken downward, so the pages cannot flatter it.

1. The micrograph

What it does. Reads the 16-bit TIFF, stretches it to 8 bit on its 0.5/99.5 percentiles, and crops the databar off: 6144×4096 px survive.

Cost. A percentile stretch, not min/max: a hot pixel or the crack itself at 0 would otherwise own the range and flatten the background texture Otsu needs downstream.

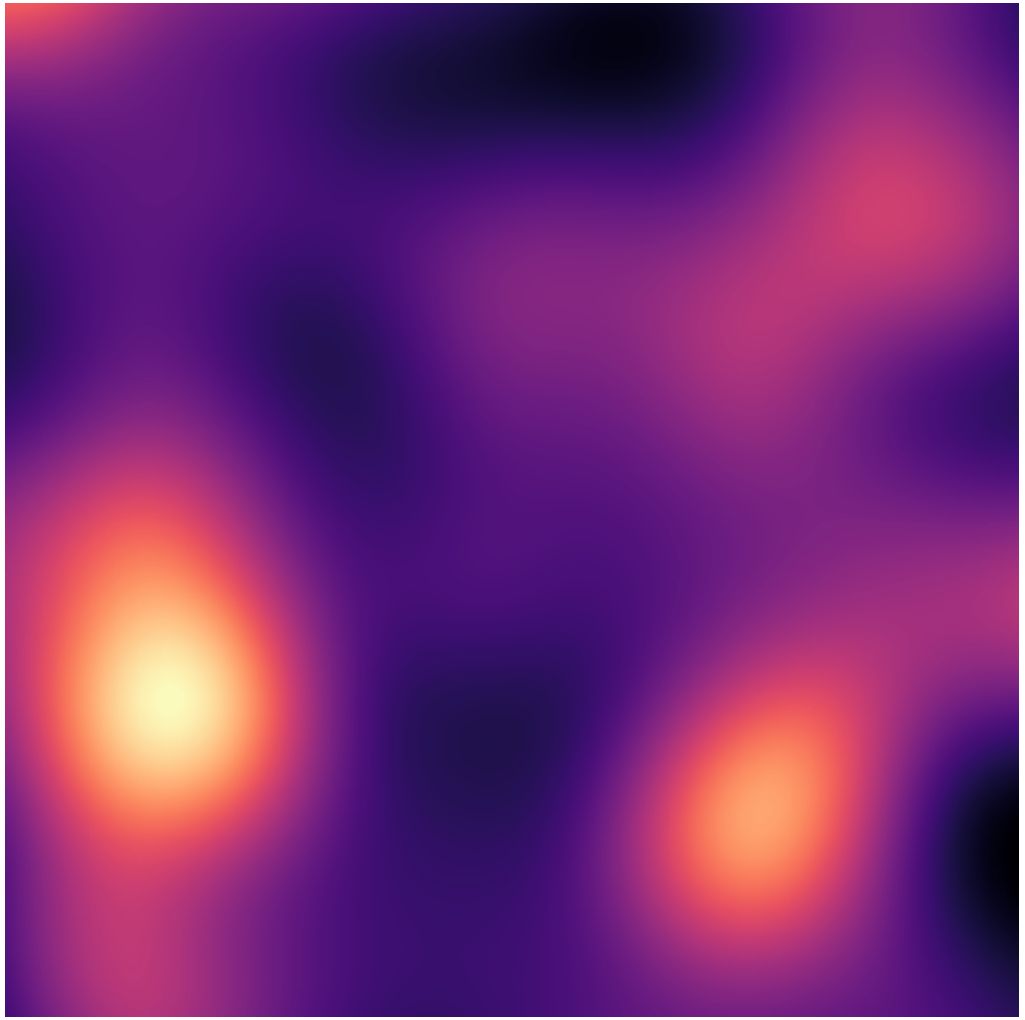


8-bit, after the 0.5/99.5 percentile stretch and the field-of-view crop.

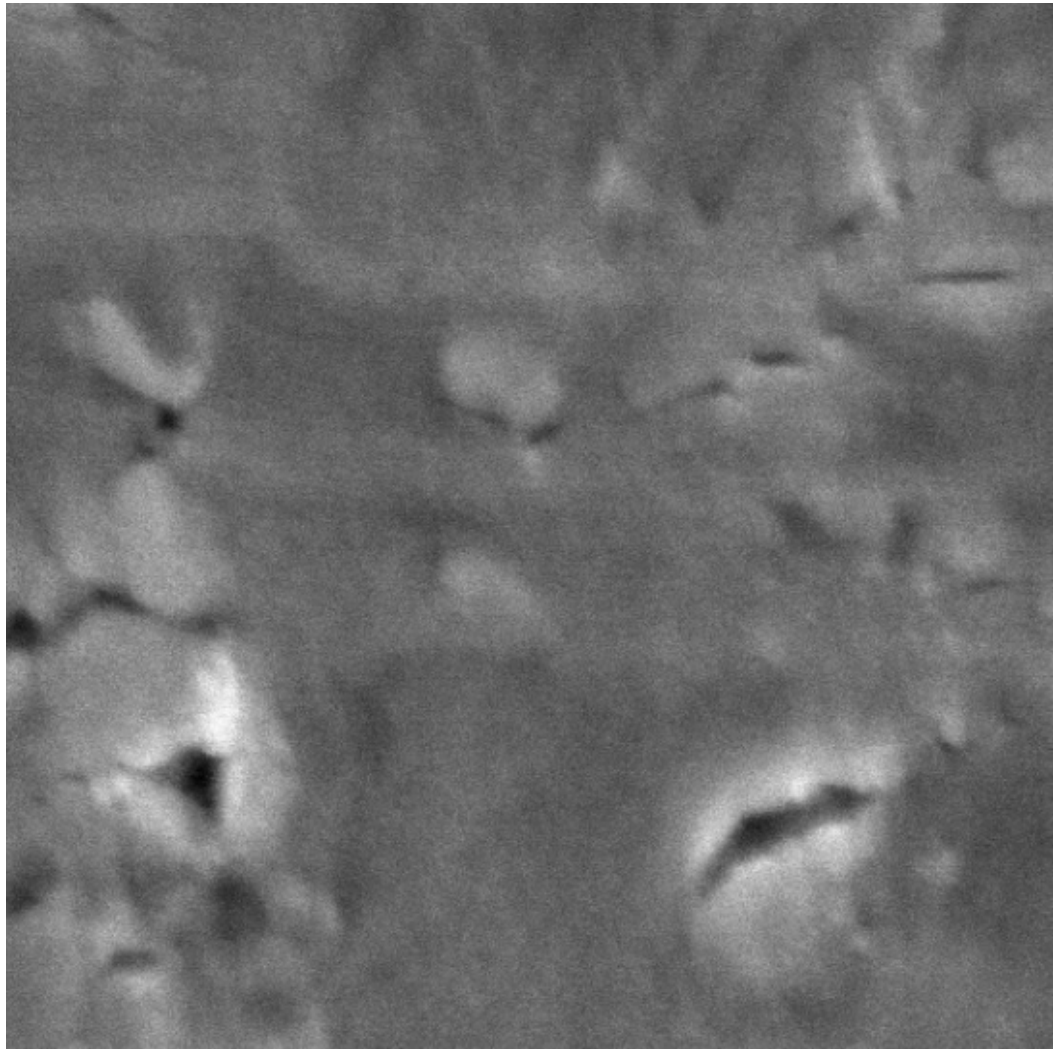
2. Why the next step exists: the illumination

What it does. Nothing yet — this is the quantity flatten_background is about to subtract.

Why it matters. A single global threshold on the raw image would track this drift instead of the crack.



The Gaussian background estimate at sigma=40 over this crop, false-coloured. It spans 92.8 to 139.3 grey levels across

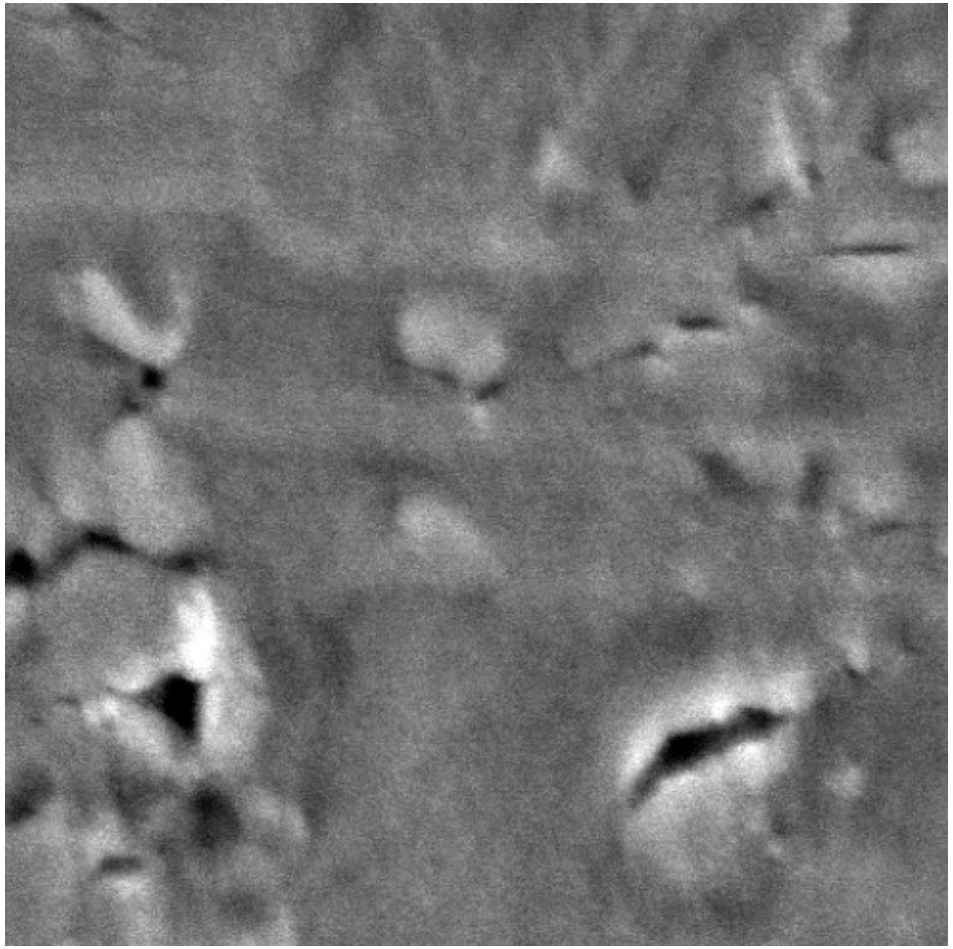


The same crop, as captured, for comparison.

3. flatten_background

What it does. Subtracts the blurred copy, so what is left is local contrast.

What it costs. It is a high-pass filter, so it also erases anything wider than its own radius. The detector's own docstring measures this: pixels that are genuinely pitch black (raw 0–4) come out near 122 of 255 after flattening, indistinguishable from mid-grey matrix. That single sentence is why steps 5 and 6 exist.



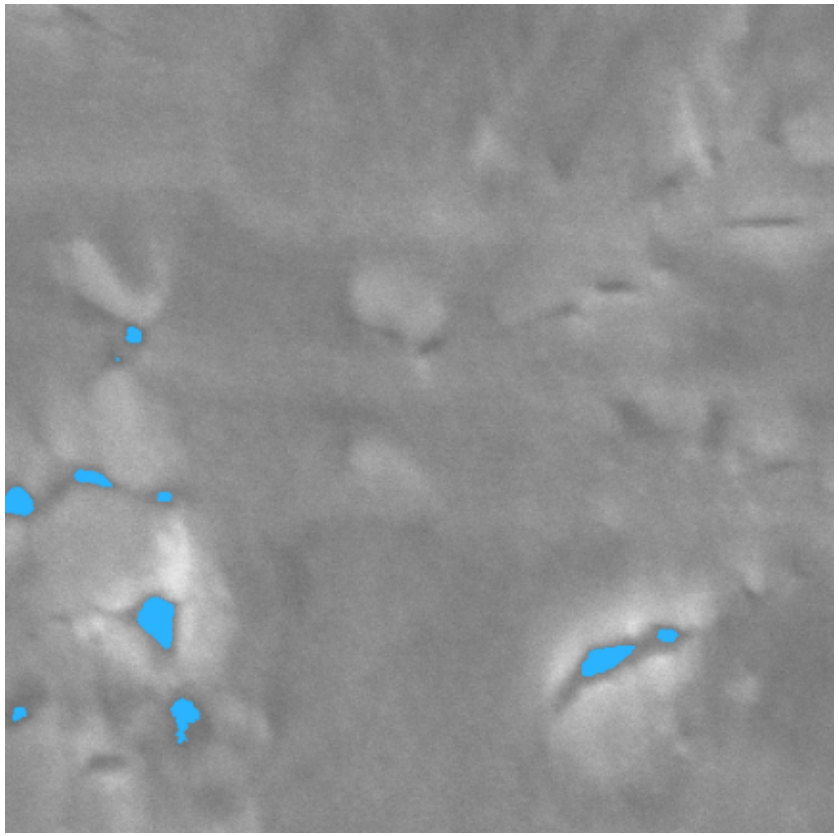
The flattened image: raw minus the background above, rescaled to 8 bit on its own 0.5/99.5 percentiles.

4. The darkness cut, at one radius

What it does. Keeps a pixel if the denoised flat image is below min(Otsu, median − 5×MAD), OR if the raw 8-bit image is below 10. Taking the lower of Otsu and the robust bar is what stops Otsu flagging half the frame on a unimodal histogram.

What it buys, on this crop. precision 0.9900, recall 0.2587, IoU 0.2580 against the hand paint. Almost everything it marks is crack; it finds about a quarter of it.

The absolute-darkness term alone carries 15 px here and 233,102 px frame-wide. At corpus scale it supplies roughly half of all the recall the detector has, and its threshold of 10 has no provenance; raising it is additive but three instruments disagree about whether to, so it stays put.

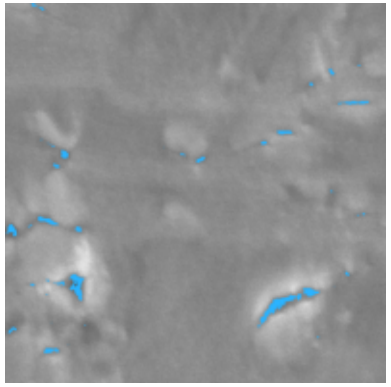


carries 1,808 px in this crop (0.69% of it) · 483,991 px in the whole frame

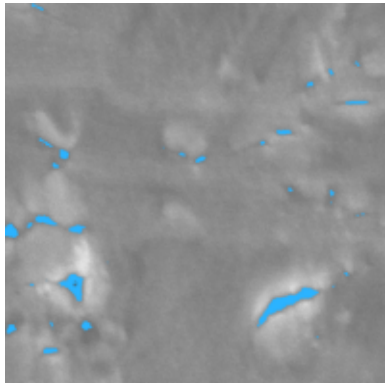
5. The same cut at each of the seven radii

What it does. Re-flattens the image at sigma = 5, 10, 20, 40, 80, 160, 320 and takes the identical cut at each. A crack is only reliably dark at the scales near its own width, so one radius sees one band.

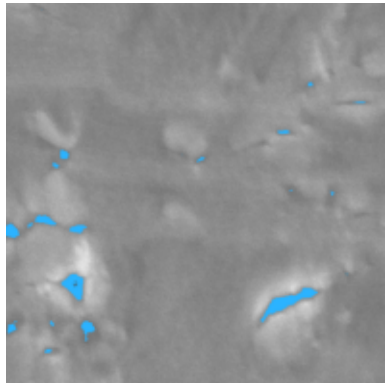
“New” counts pixels no earlier rung and not the absolute-darkness term had already claimed, taken in ascending sigma, so the seven new-counts sum to the union and no pixel is counted twice. Each rung is produced by calling the installed detector with that one rung in its ladder; the OR of the seven plus the absolute term is asserted equal, bit for bit, to the mask the pipeline itself returned.



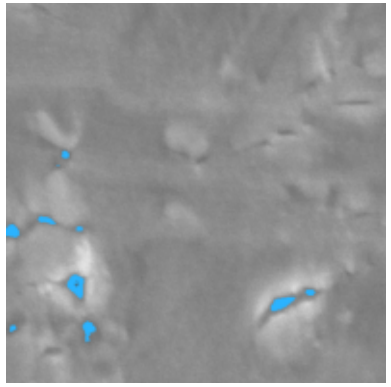
sigma = 5 · 2,155 px here · 2,155 of them new · 89,464 px frame-



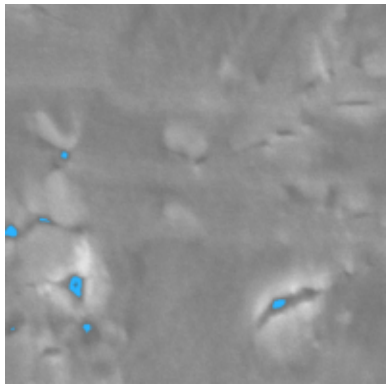
sigma = 10 · 3,456 px here · 1,382 of them new · 182,900 px



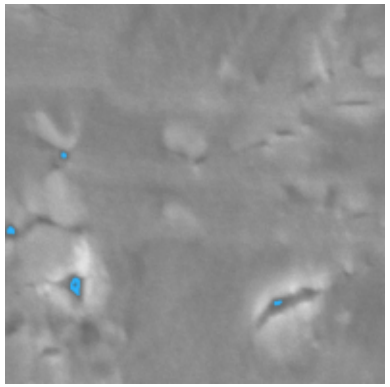
sigma = 20 · 3,012 px here · 292 of them new · 236,262 px frame-



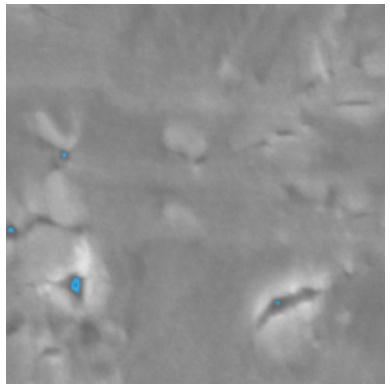
sigma = 40 · 1,793 px here · 45 of them new · 250,889 px frame-



sigma = 80 · 823 px here · 0 of them new · 266,073 px frame-wide



sigma = 160 · 400 px here · 0 of them new · 237,561 px frame-wide



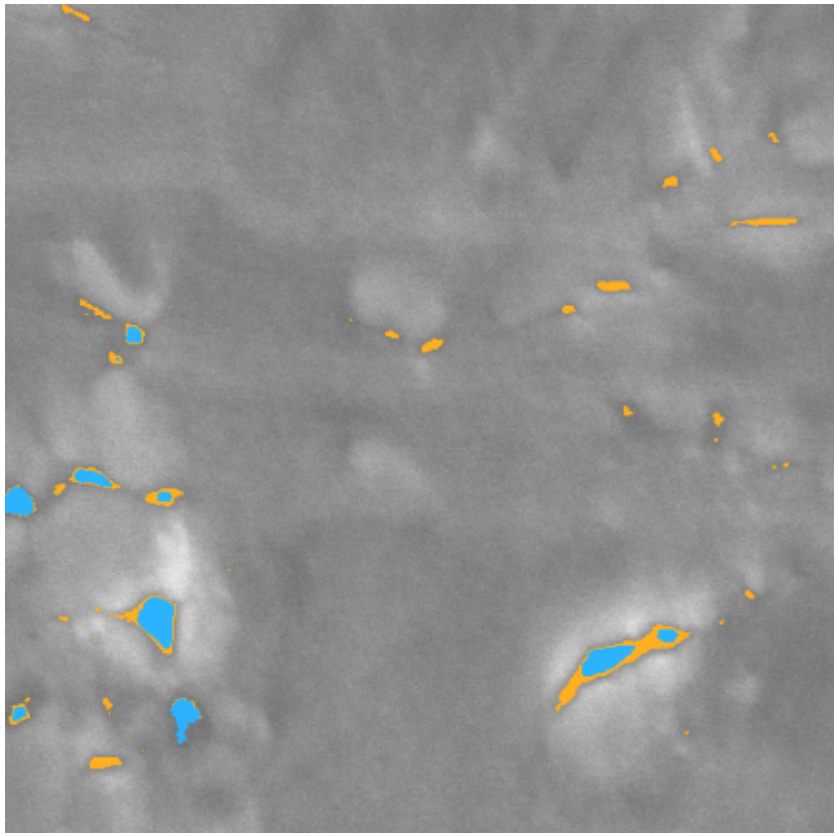
sigma = 320 · 230 px here · 0 of them new · 192,566 px frame-wide

6. Their union — the change v1.49 made

What it does. ORs the seven cuts. It is a strict superset of the single-radius mask by construction, so it cannot drop a pixel the older build kept — checked, not assumed: 0 px lost across the 26 labelled tiles and across 6,669,145 hand-painted pixels on 12 full frames.

What it buys, on this crop. +2,081 px here and +183,944 px frame-wide. precision 0.9900 → 0.8138, recall 0.2587 → 0.4574, IoU 0.2580 → 0.4141. Over the 26 labelled tiles the same change reads IoU 0.2979 → 0.3344 pooled, and it costs no new tuned constant: the ladder is every scale, not a chosen subset.

What it costs. Seven flattening passes instead of one: 12.7 s of this frame's 130.2 s. Above sigma=64 the background is estimated on a decimated copy, which is what keeps that from being 67 s.

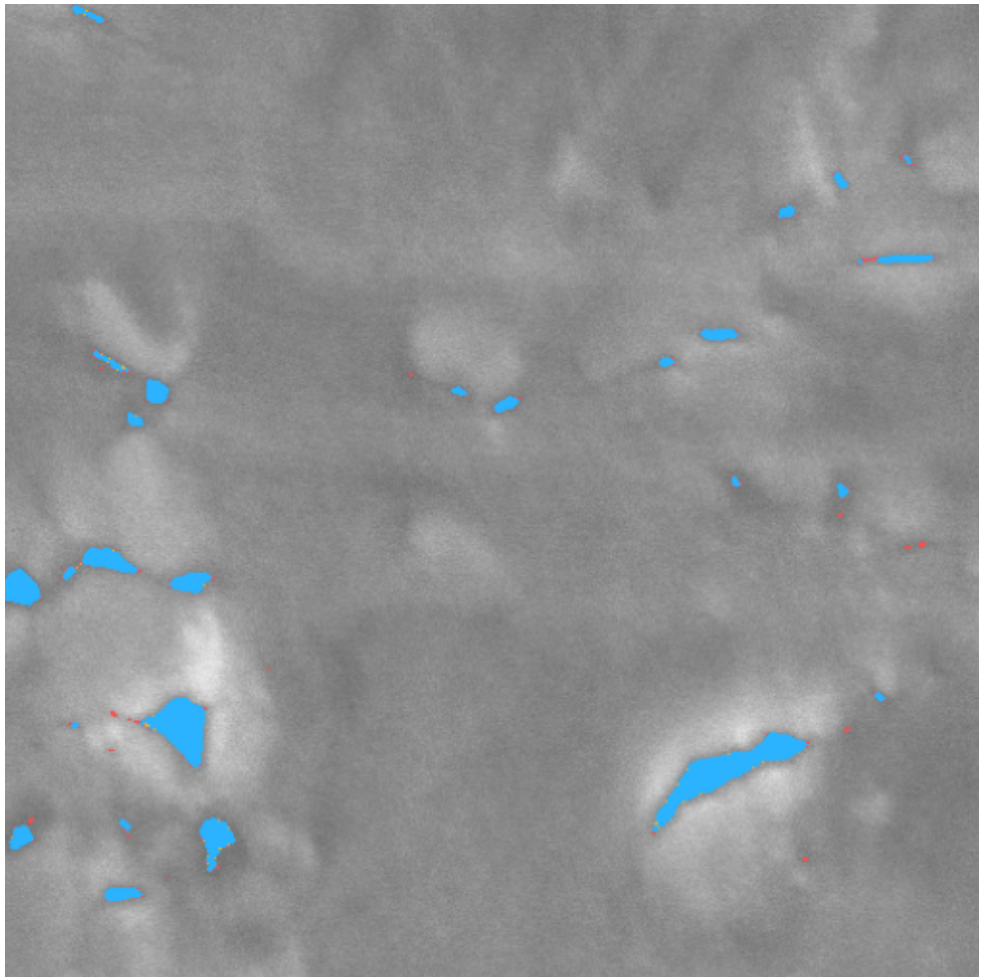


carries 3,889 px in this crop (1.48% of it) · 667,935 px in the whole frame

7. clean_mask

What it does. Drops components of 13 px or fewer, opens with a disk of radius 1, closes with a disk of radius 3, fills holes.

What it costs here. removes 168 px, adds back 42 (the closing and the hole fill are extensive), net -126 px in this crop.



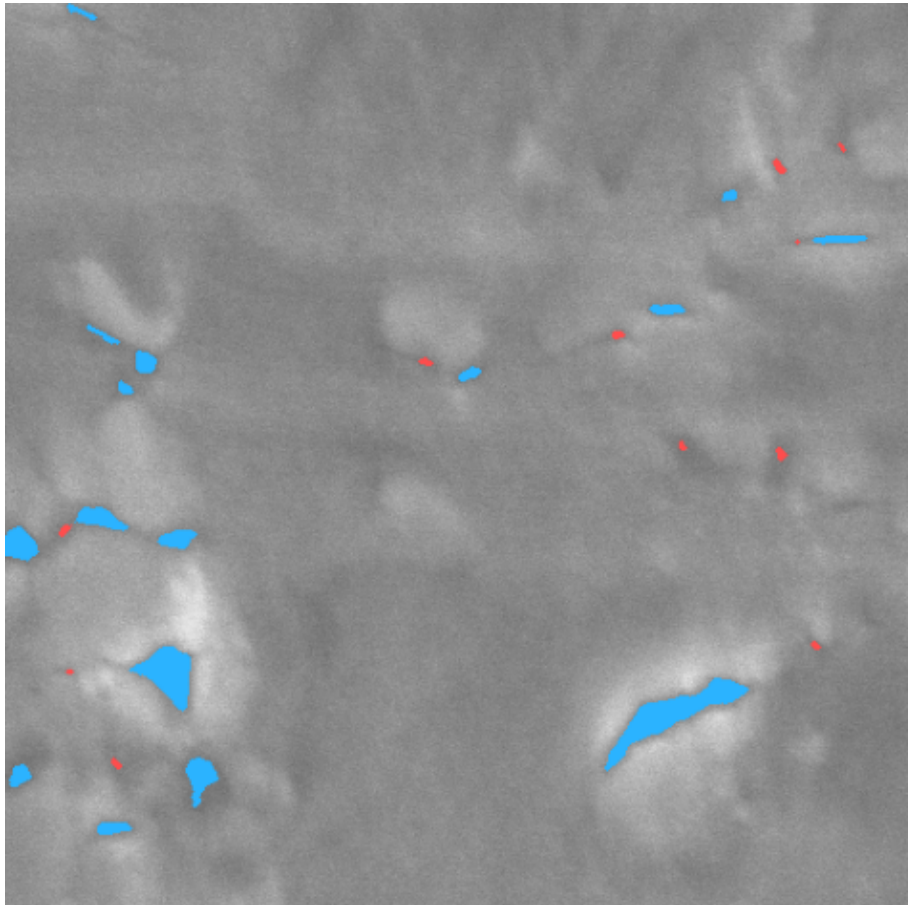
carries 3,763 px in this crop (1.44% of it) · 666,796 px in the whole frame

8. The 40 px region floor

What it does. extract_candidates labels the clean mask with connectivity=2 and keeps only components of at least 40 px, renumbering the survivors 1..N. 465 candidate regions survive on this frame.

What it costs here. 249 px in this crop.

And what does not happen next. The region classifier is USE_REGION_CLASSIFIER = False, so every one of those regions is kept. Deleting it won on all four metrics with zero free parameters; six region-level precision rules tried since all scored well on 512 px crops and then deleted 54–75% of hand-painted crack on the 48 full frames.

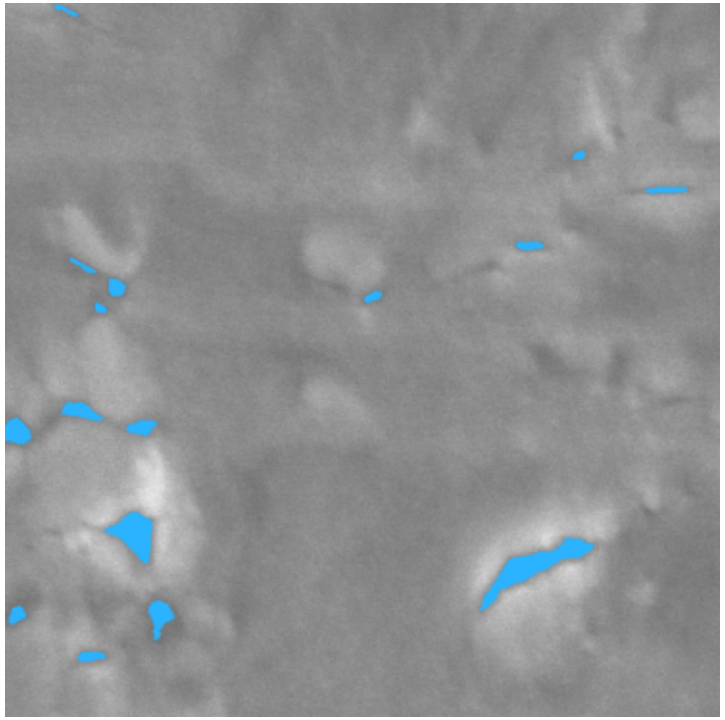


carries 3,514 px in this crop (1.34% of it) · 662,941 px in the whole frame

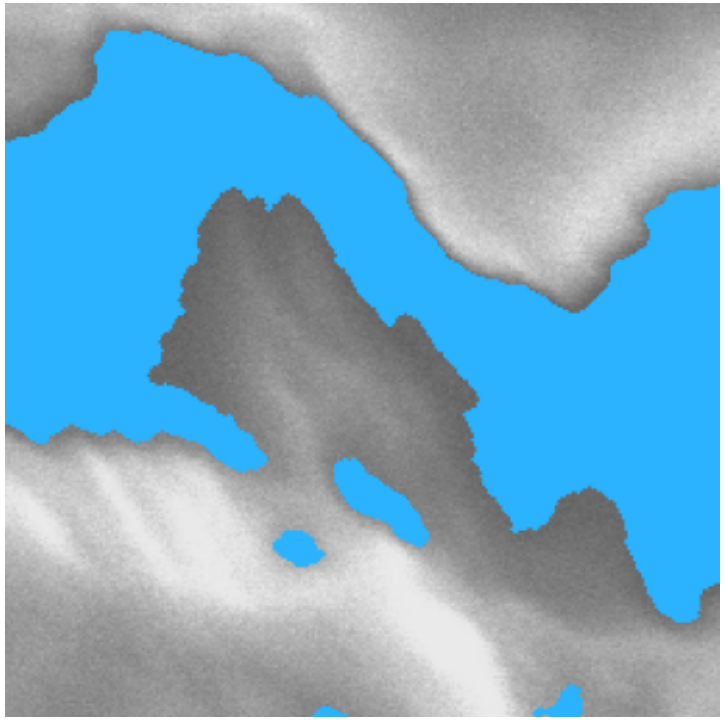
9. Pass 2

What it does. With a crack mask now in hand, proposes concavity, bridge-corridor and interior-fill candidates relative to it, scores them with the unified model, and folds the accepted ones in. 154 proposed on this frame, 129 accepted.

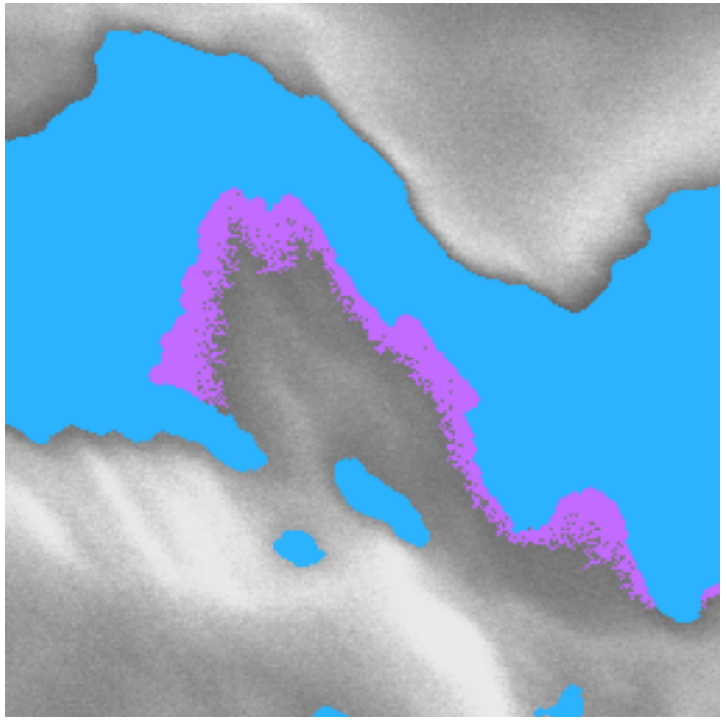
What it buys. +41,369 px frame-wide (5.87% of the final mask) and 0 px in this crop. The second pair of panels is a different place, not a different stage, and is shown because a panel of the main crop alone would read as the stage doing nothing anywhere.



carries 3,514 px in this crop (1.34% of it) · 704,310 px in



A different 256 px window, at x=0 y=992: the mask before Pass



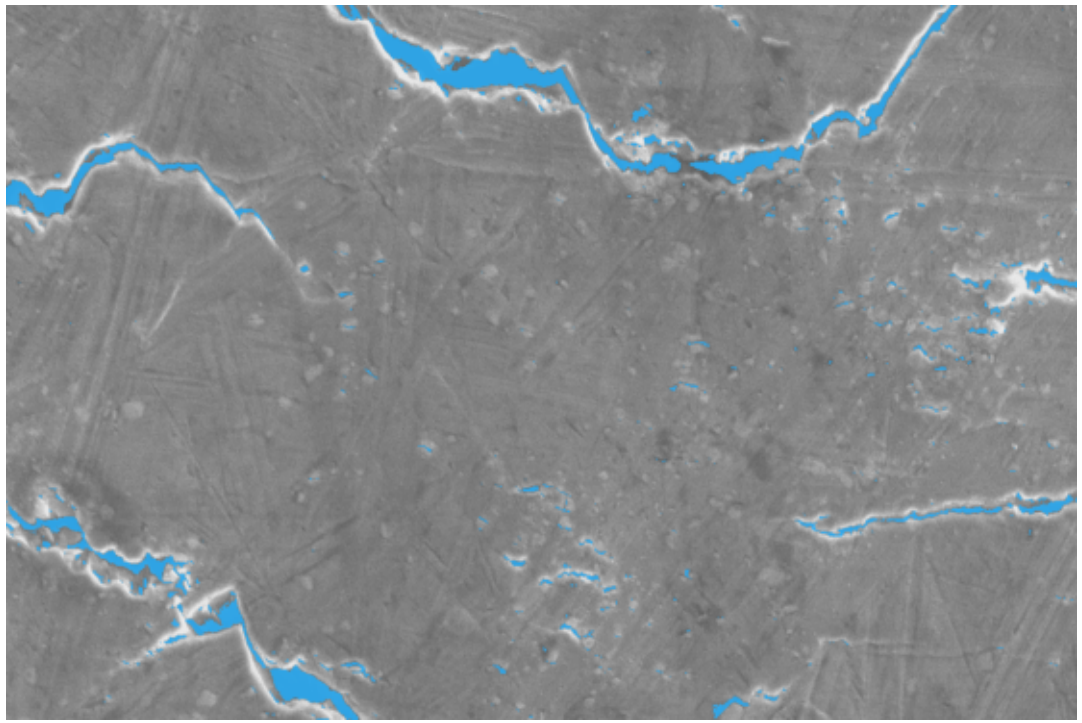
The same window after. Violet is Pass 2's addition — its largest

10. The emitted mask

What it does. Unions every kept region into one boolean mask and saves it. Area fraction 2.799% on this frame, 1.341% in this crop.



carries 3,514 px in this crop (1.34% of it) · 704,310 px in the whole frame



The same mask over the whole frame, for scale.

11. The measured output

What it does. Re-labels the emitted mask and measures every component: area, tip-to-tip geodesic, skeleton length, mean and max width, tortuosity, and whether it runs off the frame edge.

What it costs. 452 rows on this frame, 16 in this crop. Length is a bracket, not a value: 2.65% of regions on this frame touch the edge, but they are 29.57% of the crack area and 20.43% of the length. No scale bar was recovered for this frame, so every column here is pixels, not microns.



One colour per measured crack; grey is a component at or below the 25 px speck floor, which the measurement step drops.

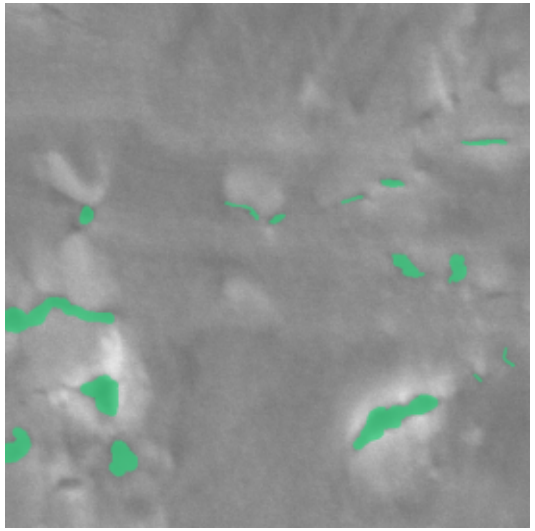
12. Against the hand paint, and where the recall goes

On this crop: precision 0.8557, recall 0.4346, IoU 0.4049. One crop is not a corpus figure.

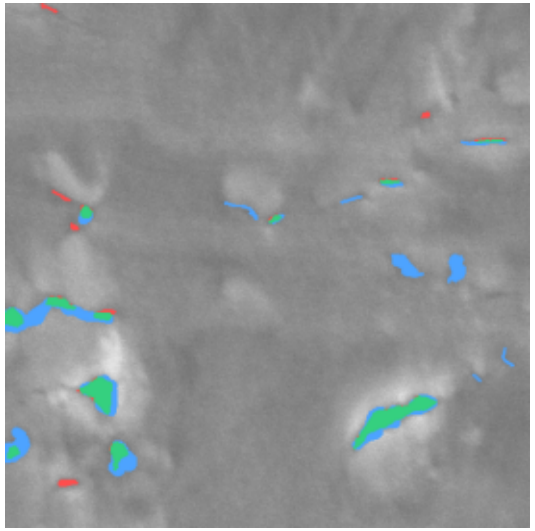
The two stages worth marking are the first row and everything else. On this crop the darkness cut owns 95.19% of all missed paint and the two morphology/area filters own 4.81% between them; over the 48 hand-painted full frames the same decomposition reads 96.02% and 3.98%. Tuning the morphology cannot reach the problem.

Reproduce every panel and every number on this page: make_flowchart.py beside this file, run under the detector environment. It asserts its own ladder against the pipeline's output and asserts the loss buckets are exhaustive before it writes anything.

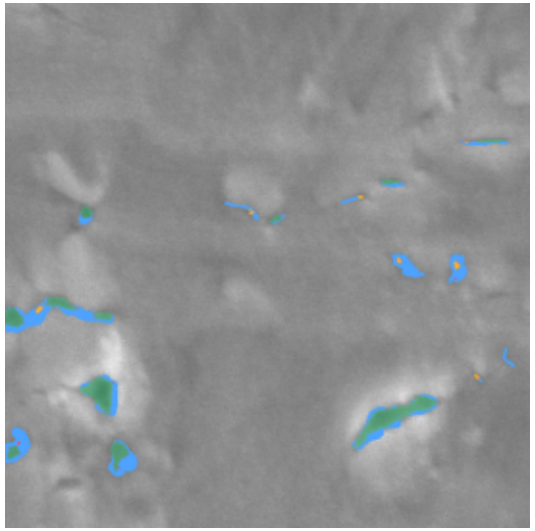
Note. How to read any number on this page. These are one crop of one frame, shown because the stages are comparable on it. The corpus figures, over the 26 hand-labelled tiles, move further with the aggregation than with any method change ever measured in this project, so a score is unreadable without its row and its tolerance:aggregationprecisionrecallF1IoUpooled over all pixels0.41820.62520.50120.3344median over the 26 tiles0.80560.76230.59220.4208median over 13 specimens0.78970.71590.54180.3715median over specimens, 2 px tolerance0.89880.80250.66330.4962A perfectly placed trace caps at IoU 0.4993 pooled and 0.5498 median over tiles on these labels, so 0.4208 is 84% of what the labels permit, not 42% of perfect. IoU and clDice are both width-gameable here — a no-information dilation to 1.5× area moves clDice +0.0478 on 13 of 13 specimens — so the metric to compare methods on is skeleton-F1 at a 2 px tolerance, where this detector scores 0.5507.



The hand paint for this crop: 6,919 px, 2.64% of it. Painted



green = found, red = marked but not painted, blue =



The missed paint, charged to the stage that dropped it: blue